

Rapport

Programmet är en interaktiv reseplanerare som är byggd med javafx. Användaren kan ladda in en bakgrundsbild som exempelvis en karta och placera ut städer som noder på kartan. Städerna kopplas ihop med kanter som representerar resvägar, där varje kant har dels ett namn(exempelvis tåg eller bil), samt en vikt som representerar restider i minuter. Användare kan ta bort städer och kopplingar, samt söka efter en väg mellan två valda städer/noder med antingen djupet- eller bredden först sökning. Grafens tillstånd kan sparas till och laddas från en textfil.

Programmet består av följande klasser:

TravelApp är huvudklassen och är den som startar javafx applikationen, den skapar en instans av TravelModel och TravelView.

TravelModel hanterar grafdata. Den använder ListGraph<String> för att lagra grafen, CityData för att lagra städernas koordinater, och PathFinder<String> för sökning.

TravelView hanterar det grafiska gränssnittet med javafx. Den har en referens till TravelModel och anropar modellens metoder för att utföra operationer.

CityData är en dataklass som lagrar ett stadsnamns koordinater på kartan.

Från grafbiblioteket används ListGraph<T> för att lagra grafen, DFSPathFinder<T> och BFSPathFinder<T> för sökning, samt GraphPath<T> för att representera hittade vägar.

Model-vy separation

Programmet har en tydlig separation mellan modell och vy enligt uppgiftsbeskrivningen. TravelModel innehåller all grafdata och programmets logik utan några javafx-beroenden. TravelView hanterar det grafiska gränssnittet och anropar modellens metoder för att utföra operationer. Detta resulterar i att logiken och presentationen i programmet är helt separerade.

Objektorienterade principer

Vårt program tillämpar flera av de olika objektorienterade principerna. Komposition används genom att TravelView har en referens till TravelModel, istället för att ärva från den. Delegering används

genom att vyn delegerar alla dataoperationer till modellen, exempelvis anropas `model.addCity()` istället för att vyn hanterar det själv. Strategimönstret används genom att `PathFinder`-gränssnittet implementeras av både `DFSPathFinder` och `BFSPathFinder`, vilket resulterar i att användaren kan byta sökalgoritm under körning utan att ändra annan kod.